Nivel 1

Descarga los archivos CSV, estudiales y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:

1) Crear BBDD sprint4:

```
create database sprint4;
```

Output

Action Output

#	Time	Action
1	19:11:06	create database sprint4

2) Crear tablas **american_users, european_users, credit_card, companies** y **transactions**:

```
use sprint4;

drop table if exists american_users;
create table american_users (
    id int primary key,
    name varchar(20),
    surname varchar(20),
    phone varchar(40),
    email varchar(50),
    birth_date varchar(20),
    country varchar(20),
    city varchar(40),
    postal_code varchar(40),
    address varchar(50),
    region varchar(20)
);

drop table if exists european_users;
create table european_users (
    id int primary key,
    name varchar(20),
    surname varchar(20),
    phone varchar(40),
    email varchar(50),
    birth_date varchar(20),
    country varchar(20),
    city varchar(40),
```

Output

Action Output

#	Time	Action	Message
6	12:41:49	drop table if exists credit_card	0 row(s) affected, 1 warning(s)
7	12:41:49	create table credit_card (id varchar(20) primary key, user_id int, iban varchar(50), pan varchar(50), pin varchar(5), cvv varchar(5), track...	0 row(s) affected
8	12:41:49	drop table if exists companies	0 row(s) affected, 1 warning(s)
9	12:41:49	create table companies (company_id varchar(50) primary key, company_name varchar(50), phone varchar(40), email varchar(50), country v...	0 row(s) affected

3) Cargar las tablas con los archivos csv y agregar datos de región en american_users y european_users para luego unirlos en una sola tabla **users** con su identificador de región.

```
load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/american_users.csv'
into table american_users
fields terminated by ','
enclosed by '"'
ignore 1 rows
(id, name, surname, phone, email, birth_date, country, city, postal_code, address)
set region = "America";

load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/european_users.csv'
into table european_users
fields terminated by ','
enclosed by '"'
ignore 1 rows
(id, name, surname, phone, email, birth_date, country, city, postal_code, address)
set region = "Europa";
```

Output

Action Output

#	Time	Action
1	19:57:22	load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/american_users.csv' into table american_users fields terminated by ',' enclosed by '"'...
2	19:57:22	load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/european_users.csv' into table european_users fields terminated by ',' enclosed by '...
3	19:57:22	load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/credit_cards.csv' into table credit_card fields terminated by ',' enclosed by '"' ignore ...
4	19:57:22	load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/companies.csv' into table companies fields terminated by ',' enclosed by '"' ignore 1 ...
5	19:57:22	load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/transactions.csv' into table transactions fields terminated by ';' ignore 1 rows

4) Crear tabla **users** con todos los datos de american_users y european_users:

```
drop table if exists users;
create table users as
select * from american_users
union all
select * from european_users;
```

Output

Action Output

#	Time	Action	Message
1	12:43:49	drop table if exists users	0 row(s) affected, 1 wa
2	12:43:49	create table users as select * from american_users union all select * from european_users	5000 row(s) affected F

5) Eliminar tablas sobrantes:

```
drop table if exists  american_users, european_users;
```

Output

Action Output

#	Time	Action
1	20:08:10	drop table if exists american_users, european_users

6) Asignar primary key a **users**:

```
alter table users
add primary key (id);
```

Output

Action Output

#	Time	Action
1	20:14:32	alter table users add primary key (id)

7) Relacionar tablas users con transactions:

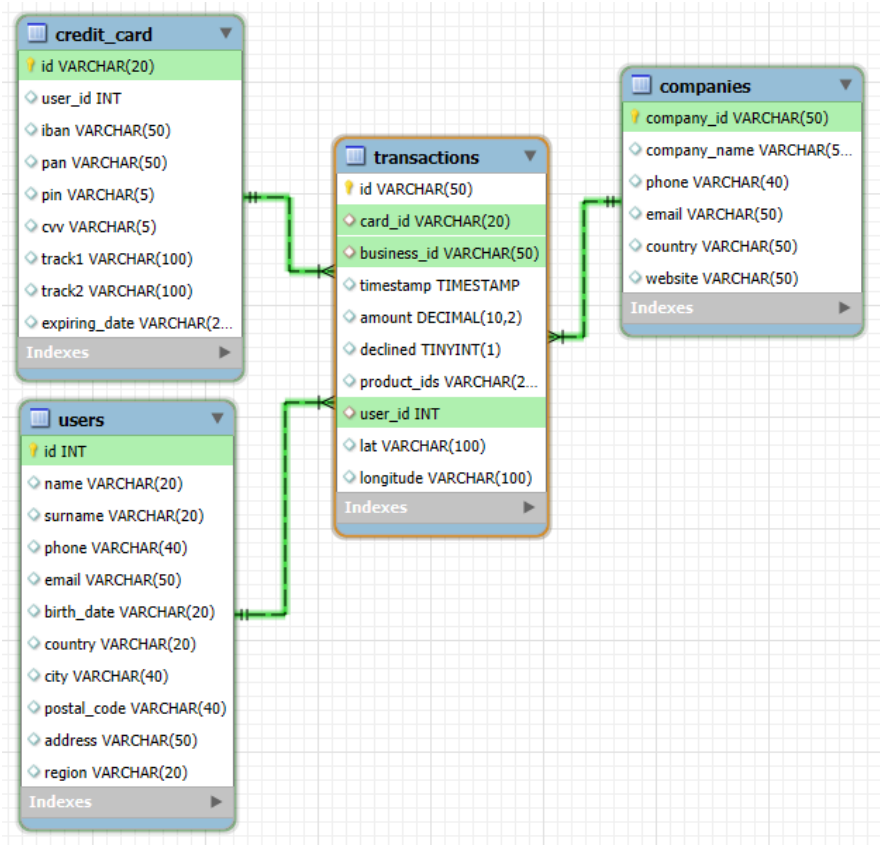
```
alter table transactions
add constraint users_transactions
foreign key (user_id)
references users(id);
```

Output

Action Output

#	Time	Action
1	20:34:43	alter table transactions add constraint users_transactions foreign key (user_id) references users(id)

8) Esquema estrella armado:



Ejercicio 1

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

```
146 •  select u.id,
147            u.name as nombre,
148            u.surname as apellido,
149            count(t.id) as cantidad_transacciones
150    from users as u
151    join transactions as t on t.user_id = u.id
152    where u.id in (select t.user_id
153                   from transactions as t
154                   where t.declined = 0
155                   group by t.user_id
156                   having count(t.id) > 80)
157    group by u.id, u.name, u.surname;
158
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	nombre	apellido	cantidad_transacciones
185	Molly	Gilliam	110
289	Dxwgi	Hwcru	94
318	Bnyr	Astuw	91

Result 1

Output

Action Output

#	Time	Action	Message
1	14:05:13	select u.id, u.name as nombre, u.surname as apellido, count(t.id) as cantidad_transacciones from users as u join transactions as t on t.user_id...	3 row(s) returned

Ejercicio 2

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.

```
162 •  select card.iban,
163            avg(t.amount) as media_ventas,
164            c.company_name as compania
165    from transactions as t
166    join companies as c on c.company_id = t.business_id
167    join credit_card as card on card.id = t.card_id
168    where c.company_name = "Donec Ltd"
169    group by card.iban
170    order by media_ventas desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban	media_ventas	compania
XX3830178139196201993663 52	680.690000	Donec Ltd
XX6377063573975703949739 13	680.010000	Donec Ltd
XX9713939714652922023122 59	645.460000	Donec Ltd
XX1718471169288923759693 07	628.890000	Donec Ltd
XX2254246388185424062235 75	608.680000	Donec Ltd

Result 8

Output

Action Output

#	Time	Action	Message
1	14:20:07	select card.iban, avg(t.amount) as media_ventas, c.company_name as compania from transactions as t join companies as c on c.company_id ...	371 row(s) returned

Nivel 2

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declinadas entonces es inactivo, si al menos una no es rechazada entonces es activo .

```
drop table if exists estado_tarjeta;
create table estado_tarjeta (
    card_id varchar(20) primary key,
    estado varchar(20)
);
```

Output

Action Output

#	Time	Action	Message
1	12:46:07	drop table if exists estado_tarjeta	0 row(s) affected, 1
2	12:46:07	create table estado_tarjeta (card_id varchar(20) primary key, estado varchar(20))	0 row(s) affected

```
insert into estado_tarjeta (card_id, estado)
select card_id,
       case when sum(declined) = 3 then 'Inactiva'
       else 'Activa'
       end as estado
from (select card_id,
             declined,
             row_number() over(
                partition by card_id
                order by date(timestamp) desc
             ) as orden
      from transactions) t
where orden <= 3
group by card_id;

select * from estado_tarjeta;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

card_id	estado
CcS-4857	Activa
CcS-4858	Activa
CcS-4859	Activa
CcS-4860	Activa
CcS-4861	Activa
CcS-4862	Activa
CcS-4863	Activa
CcS-4864	Activa

estado_tarjeta 6

Output

Action Output

#	Time	Action
1	10:16:23	insert into estado_tarjeta (card_id, estado) select card_id, case when sum(declined) = 3 then 'Inactiva' else 'Activa' end as estado from (select card...
2	10:16:24	select * from estado_tarjeta

Partiendo de esta tabla responde:

Ejercicio 1

¿Cuántas tarjetas están activas?

```
select count(estado) as cantidad_activas
from estado_tarjeta
where estado = "Activa";
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cantidad_activas
4995

Result 9

Output

Action Output

#	Time	Action
1	10:19:59	select count(estado) as cantidad_activas from estado_tarjeta where estado = "Activa"

Nivel 3

Crea una tabla con la que podamos unir los datos del nuevo archivo products.csv con la base de datos creada, teniendo en cuenta que desde transaction tienes product_ids.

1) Crear tabla **products**:

```
drop table if exists products;
create table products (
    id varchar(20) primary key,
    product_name varchar(40),
    price varchar(20),
    colour varchar(40),
    weight varchar(40),
    warehouse_id varchar(20)
);
```

Output

Action Output

#	Time	Action	Message
1	12:48:02	drop table if exists products	0 row(s) affected,
2	12:48:02	create table products (id varchar(20) primary key, product_name varchar(40), price varchar(20), colour varchar(40), weight varchar(40), war...	0 row(s) affected

2) Cargar csv:

```
load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/products.csv'
into table products
fields terminated by ','
enclosed by '"'
ignore 1 rows;
```

Output

Action Output

#	Time	Action
1	10:29:45	load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/products.csv' into table products fields terminated by ',' enclosed by '"' ignore 1 rows

3) Crear tabla puente **product_transactions** con clave primaria compuesta para permitir múltiples product_id por transaction_id y múltiples transaction_id por product_id.

```
drop table if exists product_transactions;
create table product_transactions (
    transaction_id varchar(50),
    product_id varchar(20),
    primary key (transaction_id, product_id)
    );
```

Output

Action Output

#	Time	Action	Message
1	12:49:03	drop table if exists product_transactions	0 row(s) affected, 1 v
2	12:49:03	create table product_transactions (transaction_id varchar(50), product_id varchar(20), primary key (transaction_id, product_id))	0 row(s) affected

4) Desde **transactions** insertamos los registros transaction_id y product_id los obtendremos utilizando JSON para separarlos en columnas distintas.

```
insert into product_transactions (transaction_id, product_id)
select t.id, cast(jason.value as unsigned) as product_id
FROM transactions t
join json_table(
    concat(
        '["', replace(t.product_ids, ',', '","'),'"]'
    ),
    '$[*]' columns (value varchar(50) path '$')
) as jason;
```

Output

Action Output

#	Time	Action
1	10:48:22	insert into product_transactions (transaction_id, product_id) select t.id, cast(jason.value as unsigned) as product_id FROM transactions t join json_table(

5) Relacionar tablas **product_transactions** con **products** y **transations**.

```
alter table product_transactions
add constraint pt_transactions
foreign key (transaction_id)
references transactions(id);
```

Output

Action Output

#	Time	Action
1	11:04:44	alter table product_transactions add constraint pt_product foreign key (product_id) references products(id)
2	11:04:47	alter table product_transactions add constraint pt_transactions foreign key (transaction_id) references transactions(id)

Genera la siguiente consulta:

Ejercicio 1

Necesitamos conocer el número de veces que se ha vendido cada producto.

```
272 •  select product_id, count(product_id) as veces_vendidos
273    from product_transactions
274    group by product_id;
275
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	veces_vendidos
1	2468
10	2571
100	2517
11	2573
12	2558
13	2560

Result 6

Output

Action Output

#	Time	Action	Message
1	14:13:15	select product_id, count(product_id) as veces_vendidos from product_transactions group by product_id LIMIT 0, 1000	100 row(s) returned